MP307 Practical 1 Queueing Theory I

Name: Cian Mac Liatháin

ID: 11476078

In [2]:
```
# import a standard Python package for doing matrix theory below
import numpy as np
import matplotlib as plt
```

Q.1

Consider the two state telephone example considered in lecture 2 with transition matrix

$$P = \begin{bmatrix} 1/2 & 1/2 \\ 2/3 & 1/3 \end{bmatrix}$$

This can be stored in a Python array `P` as follows:

In [3]:
```
P=np.array([[1/2,1/2],[2/3,1/3]])
print(P)
```

```
[[0.5        0.5        ]
 [0.66666667 0.33333333]]
```

$\text{Prob}(i \to j)$ in 2 time steps is found by evaluating P^2.
We may multiply two matrices using the `np.matmul` Python function.

In [4]:
```
np.matmul(P,P)
```

Out[4]:
```
array([[0.58333333, 0.41666667],
       [0.55555556, 0.44444444]])
```

We may also take the power of a matrix using the `np.linalg.matrix_power` Python function

In [5]:
```
np.linalg.matrix_power(P,2)
```

Out[5]:
```
array([[0.58333333, 0.41666667],
       [0.55555556, 0.44444444]])
```

In [6]:
```
np.linalg.matrix_power(P,5)
```

Out[6]:
```
array([[0.57137346, 0.42862654],
       [0.57150206, 0.42849794]])
```

In [7]:
```
np.linalg.matrix_power(P,20)
```

Out[7]:
```
array([[0.57142857, 0.42857143],
       [0.57142857, 0.42857143]])
```

Notice that the rows are converging which suggests that the system is ergodic i.e. for $p_m(i,j)$ the probability of transtition from state i to state j in m steps then

$$\lim_{m\to\infty} p_m(i,j) = \pi_j$$

exists for equilibrium probability π_j independent of the initial state i.

In fact the above system is ergodic with equilibrium probabilities $\pi_0 = 4/7$ and $\pi_1 = 3/7$

In [8]:
```
print(4/7)
print(3/7)
```

```
0.5714285714285714
0.42857142857142855
```

The equilibrium probabilities are found by looking for the **left** eigenvector of P for eigenvalue 1.
We can find all eigenvectors and eigenvalues of P by use of the `np.linalg.eig` Python function as follows.

In [9]:
```
eigendata=np.linalg.eig(P)
print(eigendata)
```

```
(array([ 1.        , -0.16666667]), array([[ 0.70710678, -0.6       ],
       [ 0.70710678,  0.8       ]]))
```

The output is a tuple consisting of an array and a matrix. We may unpack the tuple as follows:

In [10]:
```
X, V=eigendata
```

In [11]:
```
print(X)
print(V)
```

```
[ 1.         -0.16666667]
[[ 0.70710678 -0.6       ]
 [ 0.70710678  0.8       ]]
```

The eigenvalues of P are given in the array `X` with eigenvectors given in the matrix `V`.
In particular the eigenvector for 1 is the first column vector with equal entries. We can read this off the entries of the first column as an array:

In [12]:
```
u=V[:,0] # Note that Python indices run as 0,1, ...
print(u)
```

```
[0.70710678 0.70710678]
```

A constant scalar multiple of an eigenvector is also an eigenvector so in fact we have confirmed that $Pu = u$ for $u = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Thus the `np.linalg.eig` Python function returns the **right** eigenvectors of P.
Therefore we consider the right eigenvectors of the transpose of P, denoted by P^T, which provide us with the left eigenvectors of P.

In [13]:
```
PT=P.transpose()
print(PT)
```

```
[[0.5         0.66666667]
 [0.5         0.33333333]]
```

In [14]:

```
eigendata=np.linalg.eig(PT)
```

In [15]:

```
X, V=eigendata
print(X)
print(V)
```

```
[ 1.         -0.16666667]
[[ 0.8        -0.70710678]
 [ 0.6         0.70710678]]
```

Thus the left eigenvector of P for eigenvalue 1 is

In [16]:

```
u=V[:,0]
print(u)
print(u.real) # print real part of u
```

```
[0.8 0.6]
[0.8 0.6]
```

We normalize this eigenvector so that the entries sum to unity. We thus obtain the equilibrium probability vector π

In [17]:

```
pi=u/sum(u) # divide elements of u by sum of elements of u
print(pi.real)
print([4/7,3/7])
```

```
[0.57142857 0.42857143]
[0.5714285714285714, 0.42857142857142855]
```

Q.2

A finite queue of maximum size 3 is observed with the following transition matrix

$$\begin{bmatrix} 1/3 & 0 & 2/5 & 4/15 \\ 1/4 & 0 & 3/10 & 9/20 \\ 0 & 2/3 & 1/5 & 2/15 \\ 1/5 & 0 & 2/5 & 2/5 \end{bmatrix}$$

1. Find Prob($i \to j$ in 10 steps).

2. Find the equilibrium probabilities $\pi_0, \pi_1, \pi_2, \pi_3$.

In [18]:

```
P=np.array([
[1/3,0,2/5,4/15],
[1/4,0,3/10,9/20],
[0,2/3,1/5,2/15],
[1/5,0,2/5,2/5]
])
print(P)
```

```
[[0.33333333 0.         0.4        0.26666667]
 [0.25       0.         0.3        0.45      ]
 [0.         0.66666667 0.2        0.13333333]
 [0.2        0.         0.4        0.4       ]]
```

In [19]:
```
# 2.1 Find Prob(i->j in 10 steps)
np.linalg.matrix_power(P, 10)
```

Out[19]:
```
array([[0.17004087, 0.21052518, 0.31578941, 0.30364454],
       [0.17004046, 0.21052647, 0.315789  , 0.30364406],
       [0.17003993, 0.21052791, 0.31578988, 0.30364227],
       [0.17004086, 0.21052518, 0.31578941, 0.30364455]])
```

Repeat the sequence of steps followed Q.2.

In [20]:
```
#2.2 Find the equilibrium probabilities π0,π1,π2,π3

#Find left eigendata by first transposing matrix

PT = P.transpose()
print(PT)
```

```
[[0.33333333 0.25       0.         0.2        ]
 [0.         0.         0.66666667 0.         ]
 [0.4        0.3        0.2        0.4        ]
 [0.26666667 0.45       0.13333333 0.4        ]]
```

In [21]:
```
#Calculate left eigenvalues and eigenvectors
eigendata=np.linalg.eig(PT)
X, V=eigendata
print(X)
print(V)
```

```
[ 1.        +0.j         -0.1       +0.23804761j -0.1       -0.23804761j
  0.13333333+0.j        ]
[[ 3.30217662e-01+0.j         -2.50201620e-01-0.01669904j
  -2.50201620e-01+0.01669904j  7.07106781e-01+0.j        ]
 [ 4.08840914e-01+0.j          7.79444298e-01+0.j
   7.79444298e-01-0.j         -1.20682220e-16+0.j        ]
 [ 6.13261372e-01+0.j         -1.16916645e-01+0.27831728j
  -1.16916645e-01-0.27831728j -4.82728882e-17+0.j        ]
 [ 5.89674396e-01+0.j         -4.12326034e-01-0.26161825j
  -4.12326034e-01+0.26161825j -7.07106781e-01+0.j        ]]
```

In [23]:
```
u=V[:,0] # our unique eigenvalue of 1 corresponds to the first eigenvector

print("Equilibrium probabilities:", u.real)
```

```
Equilibrium probabilities: [0.33021766 0.40884091 0.61326137 0.5896744 ]
```

Q.3

Consider the random walk on 6 sites with the following transition matrix

$$\begin{bmatrix} 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/4 & 0 & 1/2 & 0 & 1/4 \\ 1/4 & 0 & 1/4 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \end{bmatrix}$$

Is the system ergodic? Compare your result to that for the modified random walk with transition matrix below and explain the observed difference in behaviour.

$$\begin{bmatrix} 1/4 & 1/4 & 1/2 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/4 & 0 & 1/2 & 0 & 1/4 \\ 1/4 & 0 & 1/4 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \end{bmatrix}$$

In [24]:

```
P=np.array([
[1/2,0,1/2,0,0,0],
[0,1/2,0,1/2,0,0],
[0,0,1/2,0,1/2,0],
[0,1/4,0,1/2,0,1/4],
[1/4,0,1/4,0,1/2,0],
[0,0,0,1/2,0,1/2]])
print(P)
```

```
[[0.5  0.   0.5  0.   0.   0.  ]
 [0.   0.5  0.   0.5  0.   0.  ]
 [0.   0.   0.5  0.   0.5  0.  ]
 [0.   0.25 0.   0.5  0.   0.25]
 [0.25 0.   0.25 0.   0.5  0.  ]
 [0.   0.   0.   0.5  0.   0.5 ]]
```

In [25]:

```
#Is this system ergodic?
X, V = np.linalg.eig(P)
print(X)
```

```
[ 1.00000000e+00+0.j    2.50000000e-01+0.25j  2.50000000e-01-0.25j
 -1.11022302e-16+0.j    1.00000000e+00+0.j    5.00000000e-01+0.j  ]
```

3.1 We can see that there are two eigenvalues of 1, so the matrix is not ergodic as there are different possible equilibrium probabilities depending on initial conditions

In [26]:

```
#3.2:

P2=np.array([
[1/4,1/4,1/2,0,0,0],
[0,1/2,0,1/2,0,0],
[0,0,1/2,0,1/2,0],
[0,1/4,0,1/2,0,1/4],
[1/4,0,1/4,0,1/2,0],
[0,0,0,1/2,0,1/2]])
print(P2)
```

```
[[0.25 0.25 0.5  0.   0.   0.  ]
 [0.   0.5  0.   0.5  0.   0.  ]
 [0.   0.   0.5  0.   0.5  0.  ]
 [0.   0.25 0.   0.5  0.   0.25]
 [0.25 0.   0.25 0.   0.5  0.  ]
 [0.   0.   0.   0.5  0.   0.5 ]]
```

In [27]:

```
X2, V2 = np.linalg.eig(P2)
print(X2)
```

```
[ 9.61366523e-01+0.j          1.44316739e-01+0.2769298j
  1.44316739e-01-0.2769298j -1.11022302e-16+0.j
  1.00000000e+00+0.j          5.00000000e-01+0.j        ]
```

3.2 This alternative system is ergodic since there is only a single eigenvalue of 1.

Q.4

A queue is observed over 1000 time intervals where the size of the queue after each time step is given. Construct a simple model for this queue as a Markov chain with only nearest neighbour interactions.

1. What is the expected behaviour of the queue as time continues?

2. Is the system ergodic?

In [28]:

```
qdata = [4, 5, 6, 6, 6, 7, 6, 7, 6, 5, 4, 4, 5, 6, 7, 6, 5, 4, 3, 4, 5, 6,
         2, 3, 4, 3, 2, 3, 2, 1, 1, 2, 2, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0,
         8, 7, 6, 7, 6, 6, 5, 4, 5, 4, 3, 2, 3, 2, 3, 2, 3, 2, 1, 1, 2, 3,
         6, 5, 6, 5, 4, 5, 4, 3, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0, 1, 2, 1, 0,
         3, 2, 1, 2, 3, 2, 3, 2, 1, 0, 0, 1, 1, 2, 3, 2, 1, 0, 0, 1, 0, 0,
         1, 2, 3, 4, 3, 4, 5, 6, 5, 6, 5, 6, 7, 7, 6, 5, 4, 3, 4, 3, 3, 4,
         2, 1, 0, 0, 0, 0, 1, 2, 3, 2, 2, 3, 2, 1, 0, 1, 2, 1, 0, 1, 0, 1,
         0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 2, 1, 1, 1, 0, 0, 1, 2, 3, 2, 3, 3,
         1, 0, 0, 0, 1, 2, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0,
         1, 0, 0, 1, 2, 3, 2, 3, 2, 2, 3, 4, 4, 5, 4, 3, 2, 3, 2, 3, 2, 3,
         1, 0, 1, 0, 1, 2, 3, 3, 2, 1, 1, 2, 1, 2, 1, 2, 1, 2, 3, 2, 2, 1,
         2, 2, 1, 2, 3, 4, 5, 4, 4, 5, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0, 0, 0,
         1, 2, 2, 3, 2, 3, 4, 5, 6, 5, 6, 5, 4, 5, 6, 7, 6, 5, 6, 6, 7, 6,
         4, 5, 6, 6, 5, 6, 5, 6, 5, 6, 5, 5, 5, 4, 3, 4, 5, 6, 5, 4, 3, 3,
         2, 1, 0, 0, 0, 0, 0, 1, 2, 3, 2, 3, 4, 5, 6, 7, 7, 8, 9, 8, 9, 8,
         1, 0, 1, 2, 1, 2, 3, 4, 5, 6, 7, 7, 8, 7, 8, 9, 8, 7, 6, 5, 4, 3,
         2, 3, 2, 3, 3, 2, 1, 1, 0, 1, 2, 1, 2, 2, 3, 4, 5, 6, 5, 4, 3, 2,
         3, 2, 1, 0, 0, 1, 2, 3, 4, 3, 2, 3, 2, 2, 1, 0, 1, 0, 0, 1, 0, 1,
         6, 7, 8, 8, 9, 10, 9, 10, 9, 10, 11, 10, 9, 8, 9, 10, 11, 10, 10,
         9, 8, 7, 6, 7, 6, 7, 6, 5, 4, 4, 5, 4, 3, 2, 1, 1, 0, 1, 0, 1, 0,
         0, 0, 1, 0, 0, 1, 0, 1, 2, 3, 2, 3, 2, 3, 4, 3, 2, 1, 1, 2, 1, 2,
         2, 2, 1, 0, 1, 2, 3, 4, 5, 4, 5, 4, 3, 2, 3, 4, 5, 5, 4, 3, 4, 3,
         3, 4, 3, 2, 3, 4, 3, 2, 3, 2, 1, 0, 0, 0, 0, 0, 1, 0, 1, 2, 3, 4,
         6, 5, 4, 3, 2, 1, 0, 0, 1, 0, 0, 1, 0, 1, 2, 3, 4, 3, 4, 3, 2, 1,
         4, 3, 4, 4, 5, 6, 7, 6, 7, 7, 6, 5, 4, 3, 4, 3, 2, 2, 3, 4, 3, 4,
         1, 2, 3, 2, 1, 0, 0, 0, 0, 1, 1, 2, 3, 2, 3, 2, 3, 4, 5, 6, 5, 5,
         3, 2, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 2, 1, 0, 0, 0, 1, 0, 0,
         3, 2, 3, 2, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 2, 1, 2, 1, 0, 1, 2, 1,
         0, 0, 1, 2, 2, 3, 3, 2, 3, 4, 3, 2, 1, 2, 3, 3, 2, 1, 0, 1, 2, 2,
         5, 4, 5, 4, 3, 2, 3, 4, 3, 2, 1, 2, 3, 4, 3, 2, 3, 2, 1, 0, 1, 2,
         0, 1, 2, 2, 3, 2, 1, 0, 1, 2, 3, 4, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0,
         0, 0, 1, 0, 1, 2, 2, 3, 3, 4, 3, 2, 3, 2, 1, 0, 0, 1, 2, 1, 2, 3,
         1, 2, 1, 2, 3, 2, 3, 2, 1, 1, 0]
```

In [29]:

```
N=len(qdata)
```

In [30]:

```
N
```

Out[30]:

```
1000
```

N is the number of events recorded

We can easily compute the average with the following do loop

In [32]:
```
ans=0
for i in range(1,N):
    ans=ans+qdata[i-1]
#do loop complete
average=ans/N
```

In [33]:
```
average
```

Out[33]:
```
2.718
```

Apply the simplest model of an infinite nearest neighbour queue with p_1 = prob of jumping up and p_2 = prob of jumping down. These can be estimated by counting the number of jumps up vs the number of jumps down. Consider the following do loop with if statements.

In [34]:
```
nup=0 # nup counts no of up steps
ndown=0 # ndown counts no of down steps

for i in range(1,N):
    if qdata[i]>qdata[i-1]:
        nup=nup+1
    elif qdata[i]<qdata[i-1]:
        ndown=ndown+1

# if statements complete
print(nup,"up steps", ndown,"down steps")
```

```
417 up steps 421 down steps
```

In [35]:
```
p1=nup/(N-1)
p2=ndown/(N-1)
print(p1,p2)
```

```
0.4174174174174174 0.4214214214214214
```

Q: Do you consider this to be a good estimate? What is wrong?

A: #It is not possible for the queue size to get lower when it is at 0, but your estimate doesn't take this into account

In [36]:
```
# our for loop can be adapted to take that into account
# by counting the number of steps already at 0

nup=0 # nup counts no of up steps
ndown=0 # ndown counts no of down steps
n0 = 0

for i in range(1,N):
    if qdata[i]>qdata[i-1]:
        nup=nup+1
    elif qdata[i]<qdata[i-1]:
        ndown=ndown+1
    if qdata[i-1]==0:
        n0=n0+1
print(nup, "up steps", ndown, "down steps", n0, "steps at bottom")
```

```
417 up steps 421 down steps 170 steps at bottom
```

In [37]:

```
p1 = nup/(N-1)
p2 = ndown/(N-1-n0)
print("p1:", p1,"p2:",p2)
```

```
p1: 0.4174174174174174 p2: 0.5078407720144753
```

4.1

Since p2 > p1, the expected behaviour of the queue is to grow smaller until it empties

4.2 This system is ergodic as p1/p2 is not equal to 0 or 1

Q.5

A queue is observed over 10000 one-second time intervals with data as given below in the array `qdata` .

Construct a Poisson nearest neighbour model with a single arrival and servicing pattern and hence answer the following questions:

1. What is the average time taken for 1 customer to arrive?
2. What is the average number of customer servicings per second?
3. What is your estimate for the equilibrium probability $P(n \geq 4)$, where n is the queue size in this model?
4. Suppose that two equivalent servers are introduced. What would the equilibrium probability $P(n \geq 4)$ then be?

In [38]:

```
qdata = [5, 5, 5, 6, 5, 6, 6, 7, 6, 5, 6, 5, 4, 5, 6, 6, 6, 7, 8, 7, 6, 6,
```

In [39]:

```
N=len(qdata)
print(N)
```

```
10001
```

In [40]:

```
narrive = 0
nserve = 0
n0 = 0 #empty queue, can't be served

for i in range(1, N):
    if qdata[i] > qdata[i - 1]:
        narrive = narrive + 1
    elif qdata[i] < qdata[i - 1]:
        nserve = nserve + 1
    if qdata[i-1] == 0:
        n0=n0+1

print(narrive, "arrivals", nserve, "servings", n0, "empties")
```

```
3068 arrivals 3067 servings 2178 empties
```

In [42]:
```
#5.1
average_arrive = narrive/(N-1)
print(average_arrive, " seconds")
```

```
0.3068  seconds
```

In [43]:
```
#5.2
average_serve = nserve/(N-1-n0) #taking into account times when queue is em
print(average_serve, "customers served per second")
```

```
0.39209920736384557 customers served per second
```

In [44]:
```
#5.3
traffic_density = average_arrive/average_serve
print(traffic_density)
```

```
0.7824550374959244
```

In [47]:
```
#probabiliy that n >= 4
#this is equivalent to 1 - (P(n=0) + P(n=1) + P(n=2) + P(n=3)) = 1 -(1-rho

print("Equilibrium probability P(n>=4): ", traffic_density**4)
```

```
Equilibrium probability P(n>=4):  0.37483277974222884
```

In [52]:
```
#5.4 Suppose two equivalent servers are introduced
#Then average rate of serving would increase times 3

average_serve_3 = average_serve*3
print(average_serve_3, "customers served per second with 3 servers")
```

```
1.1762976220915367 customers served per second with 3 servers
```

In [53]:
```
# New traffic density with the increased serving rate
traffic_density_3 = average_arrive/average_serve_3
print(traffic_density_3)
```

```
0.26081834583197483
```

In [54]:
```
#Prob(n >=4)?
print("Equilibrium probability P(n>=4) with 3 servers:", traffic_density_3*
```

```
Equilibrium probability P(n>=4) with 3 servers: 0.004627565182002828
```